

FedEDA: Federated Learning Framework for Privacy-Preserving Machine Learning in EDA

Joonseok Kim*, Donggyu Kim*, Seonghyeon Park, and Seokhyeong Kang
Pohang University of Science and Technology, Pohang, Republic of Korea
{adrs77, gyu511, seonghyeon98, shkang}@postech.ac.kr

Abstract—In advanced nodes, the optimization of Power, Performance, and Area (PPA) is becoming increasingly complex, requiring significant resources and time for circuit design and optimization using Electronic Design Automation (EDA). As a key approach to overcome these challenges, Machine Learning (ML) techniques have been widely studied in the field of EDA. However, security concerns around Intellectual Property (IP) limit access to real-world circuit data, making it difficult to gather sufficient data for training ML models. This lack of available circuit benchmarks restricts progress in ML research. In this study, we propose *FedEDA*, which, to the best of our knowledge, is the first Federated Learning (FL) aggregation algorithm specifically designed for EDA. *FedEDA* addresses concerns about IP security by exchanging model weights among FL participants instead of sharing raw data. Furthermore, *FedEDA* leverages Rent's Rule and circuit size to capture the hierarchical structure of circuits, mitigating issues related to data imbalance among participants and improving the quality of weight aggregation on EDA data. We demonstrate the applicability of *FedEDA* across various EDA tasks, including routability, parasitic RC, and wirelength prediction. *FedEDA* outperforms existing FL algorithms in EDA tasks, demonstrating superior performance.

I. INTRODUCTION

Recent technological advancements have significantly increased transistor integration density [1]. As chip designs become more complex, the costs and efforts in Integrated Circuits (IC) development also rise, with resource shortages for optimizing servers and license scheduling presenting significant bottlenecks in this process [2]. Machine Learning (ML) has emerged as an effective solution to these problems, significantly reducing the time required for IC design [3]–[5].

In ML, model performance typically improves with larger and more reliable datasets [6]–[8]. However, unlike famous application fields such as computer vision and natural language processing, which have large accessible datasets [9]–[11], ML in Electronic Design Automation (EDA) suffers from data scarcity. This is due to a lack of data sharing in the EDA community to protect one's intellectual property (IP) from illegal duplication [12]–[16]. Centralized data collection for ML training can lead to IP infringement, as malicious users could access the database and extract patented circuit information [17]–[19]. This security issue limits data diversity in ML for EDA (ML-EDA).

To address data privacy concerns, Federated Learning (FL) [20] was developed, focusing on data localization and

remote model training. FL is a distributed computing approach where independently trained models are aggregated without direct access to private data. In this setup, clients send their locally trained model weights to a central server, which aggregates these weights to update the global ML model. This method ensures secure ML model training without risking data leakage, as private data remains strictly within each client's local environment. However, since local clients hold data that differs in both quantity and label distribution, the issue of data imbalance arises, leading to significant performance degradation of the global model when aggregating the locally trained model weights.

P. Jingyu *et al.* introduced FLNet [21], which demonstrates the application of FL in the routability prediction task. FLNet alleviates the effects of data imbalance by reducing the number of model parameters, thus mitigating performance degradation in FL aggregation. However, it is a **model-based** approach that addresses data imbalance issues simply by modifying the model. Additionally, it does not leverage the unique characteristics of EDA data. This approach cannot integrate state-of-the-art models designed to improve prediction accuracy in the EDA domain, limiting the broad application of FL in the field.

In this work, we propose a novel model aggregation algorithm called *FedEDA*, specifically designed for EDA tasks. *FedEDA* incorporates the unique complexity and hierarchical characteristics of circuit data into the algorithm. This approach addresses data imbalance issues in distributed ML-EDA environments in the weight aggregation process, which is independent of the conventional model-based approach [21]. To demonstrate the applicability of *FedEDA*, we evaluate its performance across various prediction tasks and widely used ML models in the EDA domain. The contributions of our research can be summarized as follows:

- We present *FedEDA*, which to the best of our knowledge, is the first FL **aggregation algorithm** designed specifically for EDA. *FedEDA* addresses the challenges of data scarcity and imbalance in ML-EDA by considering the unique characteristics of EDA data in its weight aggregation process while ensuring data privacy and security.
- We recognize that circuit heterogeneity is the key factor leading to performance degradation. To capture circuit complexity, we consider Rent's Rule [22] along with circuit size. *FedEDA* systematically incorporates these characteristics into its loss function.
- *FedEDA*'s applicability is demonstrated across a wide range of EDA prediction tasks, including routability, parasitic RC, and wirelength prediction, using various models.

*These authors equally contributed to this work. This work was supported by the Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2023-00222085, Development of memory module and memory compiler for non-volatile PIM) and by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No.2019R1A5A1027055).

It outperforms conventional model merging algorithms like *FedAvg* [20], *FedProx* [23] and achieves up to an 8.74% improvement in prediction accuracy compared to FLNet in routability prediction.

II. PRELIMINARIES

A. Merging Independently Trained Models

Model merging is an ML technique that combines the experiences of independent training runs without additional training or fine-tuning. The primary concept behind this strategy is to integrate various models, each trained on local data, to enhance performance by utilizing the collective expertise of multiple models. A key benefit of model merging is its contribution to data privacy, as it allows independently trained models to be combined without the need for centralized data storage. A notable example of this approach is FL [20]. A well-known algorithm that exemplifies this approach is *FedAvg* [20], an effective FL algorithm that aggregates locally trained models at each iteration. This process can be summarized as follows:

$$w_{t,j}^k \leftarrow w_{t,j-1}^k - \eta \sum_{b=1}^B \nabla_w F(y_b, f_{w_{t,j-1}^k}(x_b)), \quad (1)$$

where B is batch size, t is global round, j is local round, k is client, and w is model weight. At every global round t , for each client $k \in \{1, \dots, K\}$, the local checkpoints $w_{t-1,0}^k$ are initialized to the aggregated result w_t . Every client updates their local checkpoints for each local step $j \in \{1, \dots, E\}$ by mini-batch gradient descent for a learning rate of η . Locally trained weights w_t^k are transmitted to the host, which is then merged to form the global model, as follows:

$$w_{t+1} = \frac{1}{K} \sum_{k=1}^K w_{t,E}^k. \quad (2)$$

Global training rounds are repeated by a specified global epoch T defined by the host. Upon convergence, the globally trained weights are broadcast to each client for local inference.

B. Related work

FedAvg is one of the simplest and most effective aggregation algorithms in FL [24] when data is Independent and Identically Distributed (IID) among clients. However, in most real-world scenarios, data tends to be heterogeneously distributed, or non-IID and training models on such data using *FedAvg* can lead to biased or sub-optimal models [25]. As a result, the local optima for each client diverges from the global optimum, a phenomenon known as *drift* in local updates. Naive aggregation of models with *FedAvg* does not always ensure that the averaged model closely approximates the global optimum which leads to performance degradation.

Alternative aggregation algorithms such as *FedProx* [23], *FedNova* [26] and *SCAFFOLD* [27] have been studied to address the challenge posed by non-IID data. *FedProx* [23] incorporates an L2 regularization term into the loss function, directly limiting the drift between the local optima and global optimum. *FedNova* [26] assumes the varying number of local

steps (τ_k) among participants, thus normalizing the local updates based on the number of preceding local updates before updating the global model. The *SCAFFOLD* [27] algorithm estimates local training divergence by subtracting the global model update direction from the local models, integrating this drift to refine local updates. However, these algorithms were developed primarily for conventional ML data, such as images and text.

P. Jingyu *et al.* designed a customized ML model called FLNet [21] to apply FL to routability prediction, an ML application in EDA. FLNet addresses the data heterogeneity issue in FL by removing batch normalization operators, proposing a model-based solution. However, this **model-based** approach cannot be applied to various ML-EDA models developed for performance enhancement, thus limiting the general application of FL in the field of EDA.

C. Rent's Rule

Rent's Rule [22], as one method to represent the characteristics of circuits, describes a power-law relationship between the number of input/output (I/O) terminals, denoted as T , and the number of gates, N , in a logic block. This relationship is expressed by the following equation:

$$T = kN^p, \quad (3)$$

where k and p are Rent's constant and exponent, respectively. These parameters are derived empirically and p is a measure of the interconnection complexity of the circuit. Its value ranges between 0 and 1; higher values mean higher complexity. Understanding the relationship between the number of gates and terminals allows designers to manage circuit complexity and connectivity more effectively, thereby enhancing overall design efficiency and performance [28].

In the context of FL for ML-EDA, leveraging Rent's Rule enables a systematic way to capture circuit complexity and integrate it into model aggregation algorithms like *FedEDA*. This helps address challenges arising from data heterogeneity, as circuit complexity significantly influences the distribution and behavior of EDA datasets.

III. PROPOSED METHOD

A. FedEDA for EDA based Federated Learning

We propose *FedEDA*, a robust model merging algorithm that accounts for the unique feature distribution of EDA data in FL. The key parameters used in *FedEDA* are described in Table I. Circuit designs typically exhibit a hierarchical structure [29], [30], where multiple small circuit modules are interconnected to form the top module, as depicted in Fig. 1 (a). Larger designs include more submodules and connections, expanding the diversity of new characteristics in the EDA feature space. For example, in Fig. 1 (b), circuit A offers a limited feature space, whereas a larger circuit C provides a significantly broader one. Since both A and C are hierarchically structured, the majority of the feature space introduced by A can also be represented by C, as illustrated by the red region in Fig. 1 (b). If local models are independently trained on A, B, and

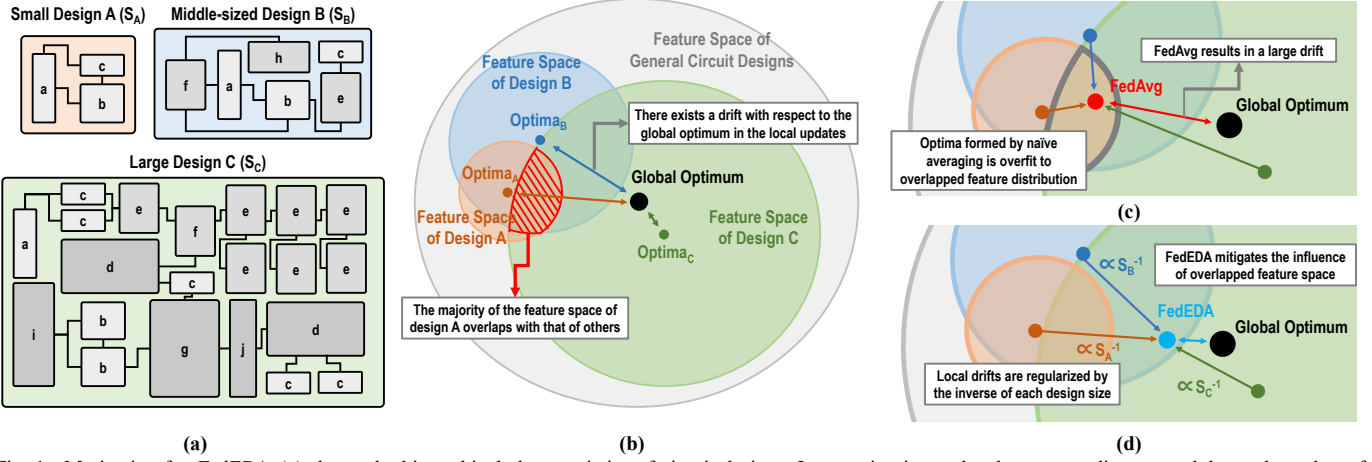


Fig. 1. Motivation for *FedEDA*. (a) shows the hierarchical characteristics of circuit designs. Larger circuits tend to have more diverse modules and number of connections. For each design i , S_i represents the circuit design size. (b) represents the feature space for designs A, B, and C. The red area on the middle left represents features shared among these three designs. (c) The averaged updates generated by *FedAvg* [20] has a considerable drift with respect to the global optima. (d) *FedEDA* alleviates this drift by taking the hierarchical information in EDA data, into account for each local update.

TABLE I
DESCRIPTION OF KEY PARAMETERS USED IN *FedEDA*

Notation	Description
p	Rent's exponent
σ	Standard deviation of p in submodule
μ	Mean of p in submodule
CM	Circuit metadata (p , σ and circuit size)
α	Inverse min-max normalization value
K	Set of clients ($k \in K$)
D^k	Local database of client k
T	Number of global iterations for FL
γ	Relative weight of importance of α
F	Original local loss function [20]
f	ML model
B	Batch size
(x_b, y_b)	Data sample from batch B

C, and averaged *FedAvg* [20], the resulting global model will be biased towards the feature space of A, since it contains the majority of the shared features, as shown in Fig. 1 (c). This significant *drift* between the averaged updates and the global optimum can lead to performance degradation when evaluated on other circuit designs. Therefore, we identify circuit size as a major factor of data heterogeneity for EDA data and incorporate this characteristic in formulating *FedEDA*.

However, circuit size alone is insufficient to capture characteristics of circuit data distribution since larger designs often consist of repetitive structures of identical submodules. We address this issue using Rent's Rule [22] by calculating Rent's parameter p for each submodule of the netlist and then computing the standard deviation σ as follows:

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (p_i - \mu)^2}. \quad (4)$$

The value of σ represents the diversity of submodules, while p indicates the complexity of the circuit. A small σ implies that the circuit contains submodules with similar characteristics, resulting in a repetitive data distribution. Similarly, a small p

indicates a simple circuit with a less diverse data distribution. From a distributed environment perspective, circuits with small σ and p contribute to a biased feature space in the overall circuit dataset. Therefore, in *FedEDA*, these factors are integrated into the local loss function, applying a penalty to circuits with smaller sizes (number of cells), low σ , and low p . The factors σ , p , and circuit size are collectively referred to as circuit metadata (CM) and represent the circuit complexity and hierarchical characteristics. CM is unique to each circuit dataset and rescaled using min-max normalization to stabilize the drift-based regularization term. Clients transmit their CM to the host and the host calculates the maximum and minimum CM values to get CM_{max} and CM_{min} , which are returned to every client. Clients calculate the inverse min-max normalization values α_{size} , α_σ , and α_p for each circuit data using the following equation:

$$\alpha = \frac{CM_{max} - CM_{min}}{CM - CM_{min}}. \quad (5)$$

As a weighted sum, α_{size} , α_σ , and α_p are combined with the L2 regularization term $\|w_t - w_{t,j-1}^k\|^2$, which represents the *drift* of the converged model and global optima [23]. The regularization term is integrated into the modified local loss function L_B^k as follows:

$$L_B^k = \sum_{b=1}^B F(y_b, f_{w_{t,j-1}^k}(x_b)) + (\alpha_{size}\gamma_{size} + \alpha_\sigma\gamma_\sigma + \alpha_p\gamma_p)\|w_t - w_{t,j-1}^k\|^2. \quad (6)$$

This approach ensures that if the local weight $w_{t,j-1}^k$ deviates significantly from the global weight w_t , the loss function increases, thus reducing the *drift* of each local update. As shown in Fig. 1 (d), the regularization of local updates per design is scaled with the inverse of its α values. Consequently, in circuit datasets where most of the features overlap with one another, the influence of local weight updates is reduced, so that the model merged by the *FedEDA* algorithm has a smaller *drift* from the global optimum.

Algorithm 1: FedEDA (Proposed Method)

```

1 Input:  $w_{t-1,0}^k, D^k, \alpha_{size}, \alpha_p, \alpha_\sigma, \gamma_{size}, \gamma_p, \gamma_\sigma, f$ 
2 Phase 1: Initialization
3   Client executes:
4     for client  $k \in K$  do
5       return  $CM^k$  to host
6     end
7   Server executes:
8      $CM_{max} \leftarrow \max\{CM^1, \dots, CM^K\}$ 
9      $CM_{min} \leftarrow \min\{CM^1, \dots, CM^K\}$ 
10    Distribute  $w_0$ , ML model,  $CM_{max}$ ,  $CM_{min}$  to  $\forall k \in K$ 
11  Client executes:
12    for client  $k \in K$  do
13      Extract  $D^k$  from Layouts
14      Initialize  $w_{t-1,0}^k \leftarrow w_t$ 
15    end
16 Phase 2: Federated learning
17 for  $t \leftarrow 1$  to  $T$  do
18   Client executes:
19     for client  $k \in K$  do
20       for  $j \leftarrow 0$  to  $E$  do
21          $w_{t,j+1}^k \leftarrow w_{t,j}^k - \eta \sum_{b=1}^B \nabla_w L_B^k$ 
22       end
23       return  $w_{t,E}^k$  to host
24     end
25   Server executes:
26      $w_{t+1} \leftarrow \frac{1}{K} \sum_{k=1}^K w_{t,E}^k$ 
27     Distribute  $w_{t+1}$  to  $\forall k \in K$ 
28   end
29 Phase 3: Application
30 Server executes:
31   Distribute  $w_{T+1}$   $\forall k \in K$ 
32 Client executes:
33   Result  $\leftarrow$  ML model(dataset,  $w_{T+1}$ )

```

B. Federated Learning Framework

The overall framework of the FL for ML-EDA is described in Algorithm 1. It involves a single administering host and multiple clients with different IC designs. This framework consists of three stages: (1) the Initialization phase, (2) the FL phase, and (3) the Application phase.

1) **Initialization phase (Algorithm 1, lines 1-15):** Initially, clients provide the FL host with CM (circuit size, p , σ) (line 5). Upon receiving metadata from each participant, the host returns processed global metadata (lines 8-9), along with design feature extractors, model architectures, and initialized weights (line 10). Each client then extracts relevant features to train their local ML models (line 13). Models for each client are initialized with the same weights to ensure consistency across participants and promote stable convergence.

2) **FL phase (Algorithm 1, lines 16-28):** After initialization, the local model is trained on the client's database with the local training epoch E and learning rate η . Initial weights w_0 are updated to w_1^k using FedEDA (line 21). Afterward, clients send their locally trained weights to the host for weight aggregation (line 23), where they are merged into the central server model (line 26). These weights are then broadcast to each client, initiating a new global round with a shared checkpoint across all participants. This global round of training is iterated for global epochs T .

3) **Application phase (Algorithm 1, lines 29-33):** In the last phase, the final weights of the converged model are distributed to each client (line 31), where they are applied to

their local ML algorithm for inference (line 33). This allows clients to predict EDA outcomes early in the physical design process, helping them assess the design validity.

Security is maintained throughout the process since no raw circuit data is exchanged. By training ML models within the FL framework, clients can enhance their models' performance by leveraging the EDA experiences of others while ensuring their own data's privacy. Furthermore, FedEDA can be integrated with complementary approaches, such as FLNet [21] or personalization techniques [31], broadening the applicability of FL in ML-EDA and enabling more versatile solutions.

IV. EXPERIMENTAL RESULTS**A. Experimental Setup**

We utilized 21 *OpenCores* [32] circuit designs as shown in Table II to train and test our ML-EDA model. We employed two commercial tools, *Design Compiler v18.06* [33] for synthesis and *Innovus v21.16* [34] for P&R. We generated 160 layouts for each circuit by sweeping the cell height, utilization, top routing layer, and P&R clock period of the layout for our experiment. Designs were processed using the *ASAP 7nm* [35] Process Design Kit (PDK) with 6-track cells and 7.5-track cells. The distributed training environment was simulated on 2.8 GHz AMD EPYC 7402P CPU with 256GB RAM, and experiments were conducted on an NVIDIA Titan RTX GPU.

B. ML Tasks for EDA

We evaluated our framework on various ML-based EDA prediction tasks, including routability, parasitic RC, and Wire-length (WL) prediction, using different ML models such as CNNs, MLPs, and GNNs. The CNN and MLP models were implemented using Python's PyTorch library, while the GNN models were implemented using the deep graph library [36]. The overall prediction task configuration and model architecture for evaluating our framework is shown in Fig. 2.

TABLE II
BENCHMARK INFORMATION

Design	6 Track			7.5 Track	
	# Ports	# Cells	# Nets	# Cells	# Nets
aes_cipher_top	391	9,205	9,464	9,376	9,635
des3	298	59,431	59,665	58,192	58,426
eth_top	211	33,021	33,277	33,658	33,914
fpu	110	16,519	18,982	18,928	21,193
jpeg_encoder	47	60,843	70,718	60,811	70,685
keccak	585	26,645	26,716	26,107	26,178
ldpc_decoder_802_3an	4,100	42,509	44,560	44,452	46,503
mc_top	267	4,070	4,208	4,073	4,211
nova	174	134,355	136,569	136,941	139,203
mpeg2_top	230	384,168	391,765	387,770	395,421
pci_bridge32	369	11,173	11,382	11,012	11,221
point_scalar_mult	932	39,422	39,963	40,460	41,002
sasc_top	28	383	399	382	398
spi_top	116	1,695	1,744	1,890	1,945
tate_pairing	1,943	167,119	167,897	169,570	170,348
tv80s	46	4,432	4,451	4,452	4,485
usbf_top	249	7,683	8,050	8,058	8,425
usb_phy	33	358	376	341	359
vga_enh_top	198	52,585	52,687	59,177	59,287
wb_conmax_top	2,546	17,744	18,874	18,249	19,379
wb_size_bridge	183	334	425	314	405

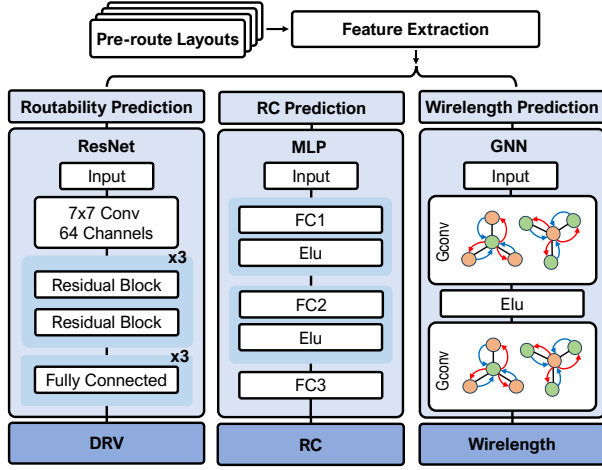


Fig. 2. ML-EDA prediction tasks to evaluate our FL framework.

We validated the applicability of our framework to ML-EDA by incorporating it into tile-level routability prediction methods [37]. Instead of directly predicting delay, we applied our approach to RC prediction, which is used for timing prediction [38], to handle extensive output slew data. Furthermore, due to the straightforward conversion of netlists into graphs, we employed GNNs, which are extensively used in the EDA domain [39], to construct the FL framework. Our EDA-based FL framework can be seamlessly applied to various state-of-the-art ML-EDA models, as demonstrated through its application to several commonly used ML models in EDA.

C. Data distribution of EDA Data

We analyzed the distribution of dataset feature space used in ML-EDA through t-SNE visualization. Here, Rent's parameters were calculated by performing min-cut partitioning on each submodule of the gate-level netlist using ML Part [40].

Fig. 3 shows the distribution of data according to circuit size, with small circuits shown in red, medium-sized circuits in blue, and large circuits in green. In Fig. 3 (a), when the standard deviation of the Rent's parameter p of the submodules is 0.05 or higher, indicating that the circuit is composed of various submodules, the feature data distribution illustrates a hierarchical structure from the lower left to the upper area according to circuit size. Conversely, Fig. 3 (b) illustrates that when the standard deviation of the Rent's parameter p is below 0.05, leading to redundant and similar submodule circuits, the feature data distribution remains clustered in the lower-left corner, regardless of increasing circuit size. Here, the standard deviation of 0.05 is the mean of the standard deviations of p for all benches used in the experiment. This highlights the hierarchical nature of EDA data, shaped by design size and complexity. By using *FedEDA* to increase the model's aggregation quality for large and complex designs, it is possible to learn from various data distributions, as shown in the upper region of Fig. 3 (a), thereby improving overall model performance.

This raises the question of whether identifying circuit complexity is meaningful for EDA datasets and how it compares to commonly used ML datasets. To explore this, we compared

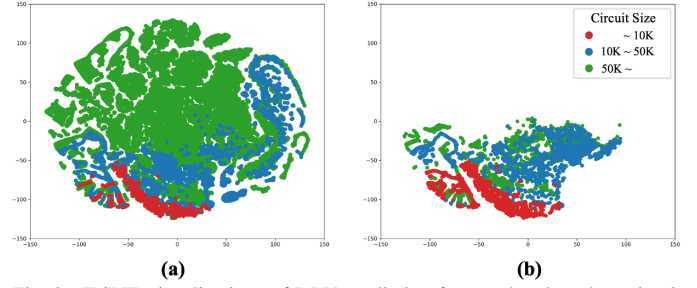


Fig. 3. T-SNE visualizations of DRV prediction feature data based on circuit size. Using a standard deviation threshold of 0.05 for the Rent's parameter p of the submodules, (a) represents designs with a higher standard deviation, while (b) represents designs with a lower standard deviation.

TABLE III
COMPARISON OF ENTROPY VALUES FOR DATASETS ACROSS DIFFERENT ML DOMAINS

Dataset	MNIST [41]	DBpedia [42]	Circuit data
Entropy	3.320	3.807	2.740

the characteristics of commonly used ML datasets with circuit data by analyzing entropy values across three datasets: the MNIST [41] vision dataset, the DBpedia [42] language dataset, and a circuit dataset used for DRV prediction. Higher entropy values indicate a greater degree of randomness in the data distribution [43]. As shown in Table III, the circuit dataset exhibits the lowest entropy value. This suggests a more concentrated distribution with less randomness, indicating that certain features are overly represented. By incorporating key factors contributing to this concentration in *FedEDA*, such as circuit size and complexity, we enhance its evaluation performance.

D. Performance Evaluation

We evaluate the effectiveness of *FedEDA* by comparing its performance against other FL methods on both classification and regression tasks. For the DRV prediction task (classification), we used accuracy and F1 score as evaluation metrics, while for the regression tasks of predicting parasitic RC and WL, we evaluated performance using the R2 score. The training dataset consisted of 18 circuits with a total of 2,280 layouts, while the test dataset included 3 circuits with a total of 480 layouts that were not used in training. For each γ value described in Eq. (6), we optimized the ideal values from the restricted candidate set $\{0.001, 0.01, 0.1, 1\}$. We tested and compared model performance with other weight aggregation techniques, *FedAvg* [20], and *FedProx* [23].

Under the IID condition, the training data was randomly shuffled and distributed in equal quantities among the clients. We adopted three different cases to emulate data heterogeneous environments. For Quantity Skew (QS), the training data was shuffled and randomly allocated to each client in unequal quantities. In the case of Label Skew (LS), the data was distributed in equal quantities to each client with no overlapping circuit designs among them. Lastly, for Label and Quantity Skew (LS+QS), the data was randomly distributed in unequal quantities with no overlapping circuit designs among the clients.

TABLE IV
COMPARISON OF PREDICTION PERFORMANCE ON VARIOUS TRAINING CONFIGURATIONS AND DATA DISTRIBUTIONS:
INDEPENDENT AND IDENTICALLY DISTRIBUTED (IID), LABEL SKEW (LS), QUANTITY SKEW (QS), AND LABEL SKEW + QUANTITY SKEW (LS+QS).

Learning Methods	Number of Clients	DRV								RC				WL			
		Accuracy				F1 Score				R2 Score				R2 Score			
		IID	LS	QS	LS+QS	IID	LS	QS	LS+QS	IID	LS	QS	LS+QS	IID	LS	QS	LS+QS
Central Learning	-	0.748				0.585				0.751				0.888			
Local Learning (Averaged)	-	0.371				0.288				0.498				0.585			
FedAvg [20]	2	0.737	0.655	0.717	0.674	0.581	0.514	0.574	0.498	0.716	0.699	0.751	0.637	0.886	0.724	0.869	0.653
	3	0.735	0.684	0.709	0.688	0.620	0.537	0.586	0.582	0.692	0.677	0.749	0.257	0.867	0.797	0.875	0.700
	6	0.731	0.630	0.726	0.690	0.533	0.500	0.539	0.464	0.765	0.764	0.795	0.388	0.881	0.791	0.887	0.527
	AVG.	0.734	0.656	0.717	0.684	0.578	0.517	0.566	0.515	0.724	0.713	0.765	0.427	0.878	0.771	0.877	0.627
FedProx [23]	2	0.733	0.703	0.715	0.711	0.568	0.443	0.496	0.559	0.763	0.731	0.753	0.725	0.884	0.811	0.886	0.802
	3	0.742	0.699	0.713	0.688	0.553	0.515	0.592	0.585	0.740	0.727	0.738	0.726	0.854	0.819	0.877	0.823
	6	0.736	0.675	0.682	0.702	0.581	0.554	0.572	0.562	0.775	0.768	0.753	0.713	0.875	0.820	0.883	0.818
	AVG.	0.737	0.692	0.703	0.703	0.567	0.504	0.533	0.569	0.756	0.742	0.745	0.721	0.871	0.817	0.882	0.814
FedEDA (Ours)	2	0.747	0.736	0.742	0.733	0.585	0.545	0.581	0.564	0.743	0.744	0.748	0.741	0.886	0.833	0.887	0.824
	3	0.748	0.733	0.739	0.731	0.582	0.554	0.574	0.586	0.742	0.743	0.749	0.746	0.886	0.857	0.882	0.841
	6	0.748	0.729	0.741	0.728	0.578	0.551	0.572	0.571	0.738	0.752	0.751	0.747	0.887	0.845	0.886	0.835
	AVG.	0.748	0.733	0.741	0.731	0.582	0.550	0.576	0.574	0.741	0.746	0.749	0.745	0.886	0.845	0.885	0.833

The best results are highlighted in **bold**.

To equate the total exposure of the dataset across the three schemes—Federated Learning (FL), Centralized Learning (CL), and Local Training (LT)—the product of the global epochs and local epochs in FL was set to be equal to the number of epochs in CL and LT per client k , as follows:

$$Epoch_{CL} = Epoch_{LT}^k = GlobalEpoch_{FL} \times LocalEpoch_{FL}. \quad (7)$$

As shown in Table IV, Local Learning, where each client trains on their own data, results in a 41.39% decrease in average performance metrics compared to the centrally trained model (Central Learning). By applying *FedEDA* in FL under the same data distribution conditions as Local Learning, the average performance metrics decrease by just 1.98%, achieving results comparable to centralized learning. Moreover, *FedEDA* provides a privacy-preserving environment with minimal impact on model performance. Additionally, compared to previous FL model merging algorithms, *FedEDA* achieves the best performance in 12 out of 12 scenarios for DRV prediction (Accuracy), 6 out of 12 for RC prediction (R2), and 12 out of 12 for WL prediction (R2), across various data distributions and client configurations. Notably, in the LS+QS distribution, which is most similar to real-world conditions, the maximum improvement in average (AVG.) F1 score and R2 score for DRV, RC, and WL prediction are 11.5%, 74.5%, and 33.5%, respectively, compared to conventional model merging algorithms [20], [23]. The γ values that yielded the best performance were 0.1 for γ_{size} , and 0.001 for both γ_{σ} and γ_p , suggesting that circuit size is the primary factor contributing to data heterogeneity in circuit data. These results demonstrate that our proposed model merging algorithm effectively addresses the data heterogeneity challenge in EDA datasets, making it highly effective and broadly applicable in ML-EDA.

E. Routability Prediction Performance Comparison

We applied our proposed *FedEDA* to a recently developed routability prediction model [37] and compared its performance with FLNet [21], since FLNet was originally targeted routability prediction. As shown in Fig. 4, *FedEDA* consistently

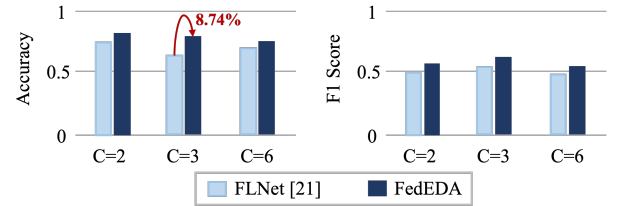


Fig. 4. Comparison of routability prediction performance between FLNet [21] and *FedEDA* in a non-IID (LS+QS) data distribution. C is the client number.

tently outperformed FLNet in both accuracy and F1 score, achieving up to an 8.74% improvement in accuracy. These findings emphasize the effectiveness of our model aggregation algorithm in overcoming the limitations of model-specific approaches, which often struggle to adapt to diverse ML models designed for performance optimization. By addressing this challenge, *FedEDA* enhances the flexibility and applicability of FL in the EDA domain. Moreover, this improvement is expected to enable *FedEDA* to support newly developed ML-EDA models, further widening the performance gap compared to model-based approaches.

V. CONCLUSION

In conclusion, our study introduces a novel FL aggregation algorithm, *FedEDA*, to address the issues of data heterogeneity and privacy concerns in the ML-EDA domain. To the best of our knowledge, this is the first work proposing an FL **algorithm** specifically for ML-EDA. Unlike previous research focused on model-based solutions [21], *FedEDA* leverages Rent's Rule and circuit size to incorporate the hierarchical characteristics of EDA data into the aggregation process, enhancing the performance of FL models and providing a more general approach. We demonstrate the effectiveness of *FedEDA* by achieving an average improvement of up to 74.5% in R2 score compared to existing aggregation algorithms. [20], [23], and up to an 8.74% improvement compared to the model-based solution, FLNet [21], across a wide range of ML-EDA tasks and various prediction scenarios.

REFERENCES

- [1] Lee C, Zhao Y, “Advanced CMOS Devices and Applications,” *Electronics*, 13, no. 1: 134, 2024.
- [2] A. B. Kahng, “Machine learning for CAD/EDA: The Road Ahead,” *IEEE Design & Test*, vol. 40, no. 1, pp. 8–16, 2022.
- [3] G. Huang, J. Hu, Y. He *et al.*, “Machine Learning for Electronic Design Automation: A Survey,” *ACM TODAES*, vol. 26, no. 5, pp. 1–46, 2021.
- [4] Z. Liu, Q. Huang, C. Fang *et al.*, “Improving test chip design efficiency via machine learning,” *Proc. ITC*, pp. 1–10, 2019.
- [5] Z. Pan, M. Li, J. Yao *et al.*, “Low-cost high-accuracy variation characterization for nanoscale IC technologies via novel learning-based techniques,” *Proc. DATE*, pp. 797–802, 2018.
- [6] A. Halevy, P. Norvig, F. Pereira “The unreasonable effectiveness of data,” *IEEE intelligent systems*, vol. 24, no. 2, pp. 8–12, 2009.
- [7] C. Sun, A. Shrivastava, S. Abhinav *et al.*, “Revisiting unreasonable effectiveness of data in deep learning era,” *Proc. ICCV*, pp. 843–852, 2017.
- [8] J. Dean, G. Corrado, C. Rajat *et al.*, “Large scale distributed deep networks,” *Proc. NeurIPS*, vol. 25, 2012.
- [9] J. Deng, W. Dong, R. Socher *et al.*, “Imagenet: A large-scale hierarchical image database,” *Proc. CVPR*, pp. 248–255, 2009.
- [10] T. Lin, M. Maire, S. Belongie *et al.*, “Microsoft coco: Common objects in context,” *Proc. ECCV*, pp. 740–755, 2014.
- [11] J. Pennington, R. Socher, C. Manning “Glove: Global vectors for word representation,” *Proc. EMNLP*, pp. 1532–1543, 2014.
- [12] X. Jiang, Y. Zhao, Y. Lin *et al.*, “CircuitNet 2.0: An Advanced Dataset for Promoting Machine Learning Innovations in Realistic Chip Design Environment,” *Proc. ICLR*, 2023.
- [13] V. Hamolia, V. Melnyk, “A survey of machine learning methods and applications in electronic design automation,” *Proc. ACIT*, pp. 757–760, 2021.
- [14] Z. Chai, Y. Zhao, Y. Lin *et al.*, “Circuitnet: An open-source dataset for machine learning applications in electronic design automation (eda),” *arXiv*, 2022.
- [15] A. Chowdhury, B. Tan, R. Karri *et al.*, “Openabc-d: A large-scale dataset for machine learning guided integrated circuit synthesis,” *arXiv*, 2021.
- [16] D. Kim, S. Lee, K. Min *et al.*, “Construction of realistic place-and-route benchmarks for machine learning applications,” *IEEE TCAD*, 2022.
- [17] K. Shamsi, M. Li, K. Plaks *et al.*, “IP protection and supply chain security through logic obfuscation: A systematic overview,” *ACM TODAES*, vol. 24, no. 6, pp. 1–36, 2019.
- [18] J. Knechtel, S. Patnaik, O. Sinanoglu, “Protect your chip design intellectual property: An overview,” *Proc. COINS*, pp. 211–216, 2019.
- [19] P. SLPSK, S. Ray, S. Bhunia, “Treehouse: A secure asset management infrastructure for protecting 3dic designs,” *IEEE Tran. Comput.*, 2023.
- [20] B. McMahan, E. Moore, D. Ramage *et al.*, “Communication-efficient Learning of Deep networks from Decentralized Data,” *Proc. AISTATS*, 2017.
- [21] Pan, Jingyu, *et al.*, “Towards collaborative intelligence: Routability estimation based on decentralized private data,” *Proc. DAC*, 2022.
- [22] Landman, Bernard S, Russo *et al.*, “On a pin versus block relationship for partitions of logic graphs,” *IEEE Tran. Comput.*, vol. 100, pp. 1469–1479, 1971.
- [23] X. Yuan, P. Li, “On convergence of FedProx: Local dissimilarity invariant bounds, non-smoothness and beyond,” *Proc. NeurIPS*, vol. 35, pp. 10752–10765, 2022.
- [24] Sannara, EK, Portes *et al.*, “A federated learning aggregation algorithm for pervasive computing: Evaluation and comparison,” *Proc. PerCom*, pp. 1–10, 2021.
- [25] Y. Zhao, M. Li, L. Lai *et al.*, “Federated Learning with Non-iid Data,” *arXiv*, 2018.
- [26] J. Wang, Q. Liu, H. Liang *et al.*, “Tackling the objective inconsistency problem in heterogeneous federated optimization,” *Proc. NeurIPS*, vol. 33, pp. 7611–7623, 2020.
- [27] S. Karimireddy, S. Kale, M. Mohri *et al.*, “Scaffold: Stochastic controlled averaging for federated learning,” *Proc. ICML*, pp. 5132–5143, 2020.
- [28] L. Hagen, A.B. Kahng, F.J. Kurdahi *et al.*, “Partitioning algorithms;Circuit synthesis;Recursive estimation;Testing;Very large scale integration;Phase estimation;Binary trees;Design automation;Computer science;Marine vehicles,” *IEEE TCAD*, vol. 13, pp. 27–37, 1994.
- [29] C. Niessen, “Hierarchical design methodologies and tools for VLSI chips,” *Proc. IEEE*, vol. 71, no. 1, pp. 66–75, 1983.
- [30] VanCleemput, W.M., “Hierarchical design for VLSI: Problems and advantages,” , 1979.
- [31] Agarwal, M., Yurochkin, M., *et al.*, “Personalization in federated learning,” *Federated Learning*, 2022.
- [32] OpenCores, *Open Source IP-Cores*, <http://www.opencores.org>.
- [33] Synopsys, *Design Compiler User Guide*, <http://www.synopsys.com>.
- [34] Cadence, *Innovus User Guide*, <https://www.cadence.com>.
- [35] L.T. Clark, V. Vashishtha, L. Shifren *et al.*, “ASAP7: A 7-nm FinFET Predictive Process Design Kit,” *Microelectronics Journal*, vol. 53, pp. 105–115, 2016.
- [36] Wang, M., Zheng, D., Ye, Z. *et al.*, “Deep graph library: A graph-centric, highly-performant package for graph neural networks,” *arXiv*, 2019.
- [37] S. Park, D. Kim, S. Kwon *et al.*, “Routability Prediction and Optimization Using Explainable AI,” in *Proc. ICCAD.*, pp. 1–8, 2023.
- [38] Chhabria, Vidya A, Jiang *et al.*, “A machine learning approach to improving timing consistency between global route and detailed route,” in *ACM TODAES*, vol. 29, pp. 1–25, 2023.
- [39] Lopera, Daniela Sánchez, Servadei *et al.*, “A survey of graph neural networks for electronic design automation,” in *Proc. MLCAD*, pp. 1–6, 2021.
- [40] Caldwell, A. B. Kahng, I. L. Markov, “Improved algorithms for hypergraph bipartitioning,” in *Proc. ASP-DAC*, pp. 661–666, 2000.
- [41] L. Deng, “The MNIST Database of Handwritten Digit Images for Machine Learning Research [Best of the Web],” *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, 2012.
- [42] S. Auer, C. Bizer, G. Kobilarov *et al.*, “DBpedia: A nucleus for a web of open data,” *International Semantic Web Conference*, Springer, Berlin, Heidelberg, pp. 722–735, 2007.
- [43] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning (Adaptive Computation and Machine Learning Series)*, vol. 1, 1st ed. Cambridge, MA, USA: MIT Press, 2017.